

Eliminating Candidates, Not Privacy

Secure Ranked Choice Voting Using MPC

Mikkel Katholm & Emil Mors

Advisors:

Sophia Yakoubov & Diego F. Aranha

January 2026

Abstract

Ranked choice voting (RCV) allows voters to express ordered preferences over candidates but is challenging to tally securely due to its iterative elimination and vote redistribution steps, which risk revealing sensitive intermediate results. As a result, many secure voting schemes avoid implementing true RCV and instead rely on scoring-based approximations.

This project presents a Secure Multi-Party Computation (MPC) protocol that faithfully implements ranked choice voting, including candidate elimination and vote redistribution, while preserving ballot privacy under a defined threat model. Ballots are encoded in a fixed-size matrix representation to enable data-independent computation, and the tally is executed through repeated MPC rounds corresponding to the standard RCV procedure. Two leakage models are considered, allowing trade-offs between efficiency and information disclosure.

The protocol is implemented in the MP-SPDZ framework and evaluated experimentally. The results demonstrate that true ranked choice voting can be computed securely using MPC for elections with a moderate number of candidates and voters, without resorting to scoring-based substitutes.

Contents

1	Introduction	1
2	Building Blocks	2
2.1	Secret Sharing Schemes	2
2.1.1	Algorithms	2
2.1.2	Properties	2
2.2	MPC	3
3	Voting Systems	3
3.1	Plurality voting	4
3.2	Ranked Choice Voting	4
3.3	Voting scheme comparison	6
4	System Architecture	6
5	Security	7
5.1	Threat Model	7
5.2	Guarantees	8
5.3	Leakage Functions	9
6	Solution Using an MPC Framework	10
6.1	Data Representation	10
6.1.1	The ballot matrix B	10
6.1.2	Active candidates vector A	10
6.1.3	Round result vector V	11
6.2	Ballot Input	11
6.3	Computation	11
6.3.1	Ballot reconstruction	11
6.3.2	Ballot counting	11
6.4	Circuit Complexity	13
6.5	Correctness of the RCV Tally	14
6.6	Minimal leakage implementation	15
6.6.1	System security of minimal leakage	15
6.7	Round based leakage	16
6.7.1	System security of more leakage	16
6.8	Why We Chose MP-SPDZ	17
7	Experimental Results	17
7.1	Varying number of voters	17
7.2	Varying number of candidates	18
7.3	Varying number of servers	18
8	Limitations and Future work	20
A	Implementation code	24

1 Introduction

In any democratic system, it is necessary from time to time, to organize elections for representatives in parliaments, councils or other assemblies. Throughout the history of democracy, numerous voting systems have been devised, each attempting to balance fairness, expressiveness, security, and practicality. While no system is perfect, the choice of voting method can greatly influence both the perceived legitimacy of the outcome and the incentives faced by voters.

Much research has been conducted into creating secure electronic voting schemes for plurality voting. However, very limited research has been conducted into creating similar schemes for Ranked Choice Voting (RCV). Within the family winner-takes-all electoral systems, plurality voting remains the most widely used due to its ease of understanding and implementation. The simplicity does however come at a cost. One well known flaw of plurality voting, is that it encourages the use of strategic voting, wherein a voter misrepresents their true preference because they believe that their true preference has little to no chance of winning. Though it does not compromise the correctness of an election, it does however increase the risk of the voters' true preference not being represented. RCV offers an alternative to plurality voting, allowing voters to rank the candidates in order of preference. Bartholdi and Orlin [1] showed that RCV drastically reduces the incentives for voters to employ strategic voting, and thus increases the likelihood that the result of the election reflects the voters' true preference. Hausladen et al. [2] showed that RCV is perceived as more fair and as being a better reflection of the voters' opinions than plurality voting.

In recent years, secure electronic voting has increasingly been studied through the lens of Secure Multi-Party Computation (MPC). MPC is a cryptographic system that allows a set of parties to jointly compute a function over their private inputs, while ensuring that no party learns anything beyond what can be inferred from the final output. In the context of voting, MPC enables the tallying of votes without ever revealing individual ballots. Security for a voting system typically encompasses several key properties: ballot secrecy, ensuring that no one can link a voter to their vote; correctness, guaranteeing that the announced outcome corresponds to the submitted ballots; integrity, preventing votes from being altered, added, or removed; and verifiability, allowing voters or external auditors to gain confidence that the election was conducted correctly. Together, these properties aim to ensure both the technical soundness of the election and the trust of the electorate in the legitimacy of the result.

Related Work: Some recent research [3, 4] claims to securely implement the ranked choice voting scheme electronically. However, these papers do not implement the RCV scheme, but rather the [Borda Count scheme](#).

Our Contributions: In this project we extend the study of privacy-preserving electronic voting by providing a secure protocol for RCV. We have designed a precise and implementation-ready specification for RCV-tallying suitable for use

with MPC. Unlike prior work, which frequently substitutes RCV with Borda Count, our design makes no such substitution and fully includes the elimination structure of RCV, including ballot redistribution and majority detection. Furthermore, we have implemented the scheme using two different leakage functionalities: one where only the winner is revealed, and one where the vote distribution after each round is revealed. Finally, we compare the protocols implementing the two leakages in terms of their running times and communication complexity.

2 Building Blocks

This section introduces the cryptographic primitives that form the foundation of our system. In particular, we rely on secret sharing schemes to distribute sensitive values among multiple parties, and on MPC to perform computations directly on these distributed values while preserving their confidentiality.

2.1 Secret Sharing Schemes

A secret sharing scheme is a method to share a value among some number of parties. A secret sharing scheme consists of the algorithms described in section 2.1.1, who together achieve the properties in section 2.1.2.

2.1.1 Algorithms

***genShares*(m, r):** This algorithm takes as input a secret message m and randomness r , and generates as output n shares s_i , which are distributed among the participating parties.

$$\text{genShares}(m, r) \rightarrow S = \{s_1, s_2, \dots, s_n\}$$

***reconstruct*(S):** The reconstruction algorithm takes at least t shares where t is the reconstruction threshold and outputs a message m'

$$\text{reconstruct}(S) \rightarrow m'$$

2.1.2 Properties

Correctness: For any message encoded into shares, running the reconstruction algorithm on enough of those shares, it holds that the reconstructed message is the same as the one encoded using *genShares*.

$$\begin{aligned}
& \forall m, \forall r \\
& \forall Q \subset [1, 2, \dots, n] \text{ s.t. } |Q| \geq t \\
& \text{genShares}(m, r) \rightarrow S = \{s_1, s_2, \dots, s_n\} \\
& \text{rec}(S_Q) = m' \\
& \text{such that } m = m'
\end{aligned}$$

Privacy For any two messages secret shared using the *genShares* algorithm, if you have access to fewer shares than the reconstruction threshold t , then the two sets of shares are indistinguishable.

$$\begin{aligned}
& \forall m, m' \\
& \forall Q \subset [1, 2, \dots, n] \text{ s.t. } |Q| < t \\
& S \leftarrow \text{genShares}(m, r) \\
& S' \leftarrow \text{genShares}(m', r') \\
& S_Q = \{s_i\}_{i \in Q}, S'_Q = \{s'_j\}_{j \in Q} \\
& \forall x \\
& \Pr_r[S_Q = x] = \Pr_{r'}[S'_Q = x]
\end{aligned}$$

2.2 MPC

MPC is a cryptographic technique that enables a set of parties to jointly evaluate a function over their private inputs without revealing those inputs to one another. Each party learns only the final output of the computation and nothing more about the individual inputs than what can be inferred from that output. MPC is particularly useful for computations involving sensitive data, such as health records, financial information, or voting ballots, where confidentiality must be preserved throughout the computation. Depending on the protocol, the output of the computation can be reconstructed only if a sufficient number of participating servers cooperate, ensuring that a limited number of corrupted servers cannot learn the private inputs or influence the result.

3 Voting Systems

One of the major electoral system is the winner-take-all system, an often rather simple and straightforward system. As the name implies, this model selects a single candidate as the sole winner, and all remaining candidates are considered unsuccessful. Winner-take-all systems are widely used, including in countries such as the United States and France, and are typically associated with plurality or majority voting rules. It is this system that we will be investigating further in this project.

3.1 Plurality voting

In this type of system each voter selects a single candidate, and the candidate receiving the highest number of votes is declared the winner. While the simplicity makes it easily understandable, it has several shortcomings.

Advantages:

- **Simplicity:** The system is easy to explain to the general public, ballots are straightforward to complete, and tallying can be performed without sophisticated infrastructure.

Disadvantages:

- **Strategic voting:** Voters who believe their preferred candidate has little chance of winning, may instead cast their vote for a candidate perceived as more likely to win. As a consequence, the elected candidate might not represent the true collective preference of the electorate.
- **Limited expressive power:** Plurality voting restricts voters to choosing a single candidate, preventing them from expressing nuanced preferences or ranking candidates by preference intensity.
- **Election of minority-supported candidates:** When more than two candidates run, the winning candidate may be supported by only a plurality, not a majority, of voters. This can result in outcomes that do not reflect the preference of the majority.
- **Perceived wasted votes:** Votes for candidates who do not win do not directly contribute to the election outcome. While this is not unique to plurality voting, other systems can also have non-winning votes, the lack of influence of these votes in plurality can encourage strategic behavior to avoid "wasting" a vote.

3.2 Ranked Choice Voting

RCV is, just like plurality voting, a winner takes all election scheme. In RCV, the voters rank the candidates in order from most desirable to least desirable. The tallying of votes then happens as follows:

1. Distribute votes according to the highest priority candidate on the ballots.
2. Check whether any candidate has a majority.
 - (a) **Yes:** Output that candidate as the winner.
 - (b) **No:** Proceed to step 3.
3. Check whether only one candidate remains.

- (a) **Yes:** Output the remaining candidate as the winner.
 - (b) **No:** Proceed to step 4.
4. Eliminate the candidate with the lowest number of first-priority votes. For each ballot assigned to the eliminated candidate, reassign it to the highest-ranked candidate that has not yet been eliminated. If there are no more candidates on the ballot, discard the ballot.
 5. Proceed to step 2.

Compared to plurality voting, RCV has both advantages and disadvantages. While plurality voting is valued for its straightforward design and ease of implementation, this simplicity also limits its ability to accurately capture voter preferences in elections involving more than two candidates. RCV addresses many of the shortcomings found in plurality voting by allowing for greater expressiveness and by producing outcomes that are often perceived as fairer [2]. However, it also introduces an additional layer of complexity, both in the tallying process and for the voter, who must now determine not only their first choice but also their second, third, and subsequent preferences. The primary advantages and disadvantages are as follows.

Advantages:

- **Reduced vote splitting:** Two similar candidates do not split the voters.
- **Broader support:** Assuming voters rank all candidates they find acceptable, the winning candidate is more likely to be acceptable to a larger fraction of voters than under plurality voting.
- **Expressiveness:** Voters can rank candidates instead of only choosing one, so it is possible to express more nuanced opinions.
- **Computational resistance to strategic voting:** While no non-dictatorial voting rule is immune to strategic manipulation, determining a beneficial strategic vote under RCV is NP-complete, making such manipulation computationally difficult in practice [5].

Disadvantages: As one might expect, the advantages comes at the cost of complexity, both in terms of the tallying proccess but also for the voters.

- **More complex system:** The scheme is not as simple as plurality voting, thus making it more difficult to understand, particularly for people with low knowledge of voting systems.
- **Tallying complexity:** The tallying process is more complex then simple plurality.
- **Complex decision process:** Voters do not just need to find their favorite candidate but also find all runner ups.

Property	Plurality Voting	Ranked Choice Voting
Voter Decision Complexity	Lower	Higher
System Complexity	Lower	Higher
Strategic Voting	Yes	No
Expressiveness	Lower	Higher
Wasted Votes	More	Fewer

Table 1: Summary of properties of Plurality Voting and Ranked Choice Voting

3.3 Voting scheme comparison

The differences between plurality voting and ranked choice voting discussed in the previous sections are summarized in Table 1. The table provides a high-level comparison of the two schemes with respect to voter experience, system complexity, and strategic considerations.

4 System Architecture

Our system consists of two principal types of participants: voters and tabulation servers. In our proof-of-concept implementation, we assume that the system begins with a set of well formatted secret shared ballots already distributed among the servers. That is, the voters have already interacted with a client application that formats their ballot and secret shared it to the servers. Below, we describe the intended architecture in a real world deployment.

Voters: Voters are the physical individuals participating in the election. They express their preferences by interacting with a client application, but do not directly participate in the cryptographic protocol. We denote the number of voters as n .

Clients: Clients are software applications that act on behalf of voters. A client guides the voter in specifying a ranking over the candidates and constructs a well-formed ballot according to the voting scheme. The client is responsible for secret sharing the ballot and transmitting the shares to the tabulation servers over secure channels. All computations related to ballot construction are performed locally by the client. Depending on the protocol variant, clients may participate in one or more rounds of interaction, but they do not take part in the tallying of votes.

Tabulation Servers: The servers are responsible for the counting of the ballots submitted by the voters. The servers do not vote themselves, but rather act in role of election officials. The servers collectively execute the tallying protocol. When the servers have received all the secret shared ballots from the clients,

they run the MPC protocol designed for tallying the ballots. The servers compute over secret shared data and do not learn anything the leakage profile does not allow.

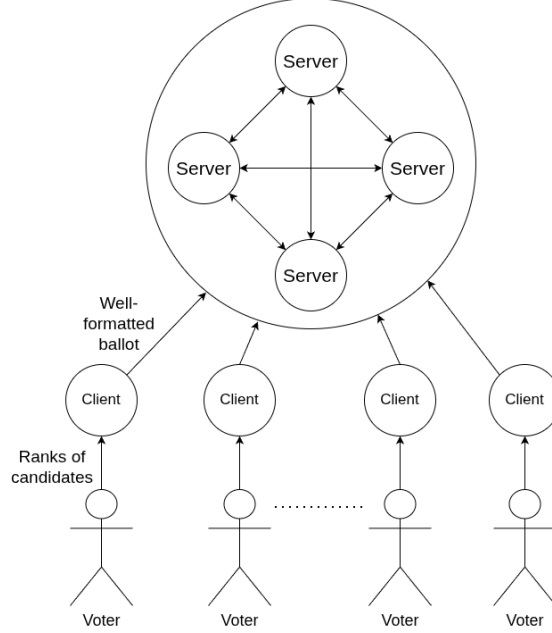


Figure 1: The overall architecture of the election system. Note that there may be an arbitrary number of servers.

5 Security

5.1 Threat Model

We consider a standard adversarial model for MPC based electronic voting systems. The system consists of a set of voters, client applications, and a fixed number of tabulation servers that jointly execute the tallying protocol.

Adversarial servers: We assume that the adversary may corrupt and fully control a subset of the tabulation servers, the subset of corrupted servers can be as large as $(k - 1)$ where k is the number of servers. Corrupted servers may deviate arbitrarily from the protocol, attempt to infer information about individual ballots, or collude with one another. Security is guaranteed as long as the number of corrupted servers remains below the threshold required by the underlying MPC protocol.

Honest voters and clients: Voters and their corresponding client applications are assumed to be honest with respect to ballot formation. In particular, clients correctly encode the voter’s intended ranking into a well-formed ballot and secret shares it according to the specified scheme before submission. Attacks involving malformed ballots, malicious clients, or coercion of voters are considered out of scope for this work.

Network communication: We consider a passive network adversary capable of observing all communication. To protect against such an adversary, all messages are transmitted over authenticated and encrypted channels, ensuring confidentiality and integrity of the communication. As a result, the adversary may observe traffic patterns but cannot learn message contents or modify protocol messages.

Adversarial goals: The primary goals of the adversary are to violate ballot privacy by reconstructing or inferring individual ballots, or to influence the outcome of the election by manipulating the tallying process. Preventing denial-of-service attacks and ensuring availability are not considered in this project.

5.2 Guarantees

While there are no universal rules for which properties an election must satisfy, several guarantees are widely considered desirable. In particular, correctness is essential; however, correctness alone is not sufficient to ensure a legitimate election. International standards also emphasize ballot secrecy: Article 25 of the International Covenant on Civil and Political Rights states that elections “shall be held by secret ballot, guaranteeing the free expression of the will of the electors” [6].

Correctness: The output of the system must reflect how the votes were cast. That is, the winner must be determined according to the predetermined rules of the system.

Ballot Privacy: Ballot privacy, also known as secret ballot, requires that no one but the one who cast the ballot learns any information about their ballot, beyond what is implied by the final election output as well as any explicitly permitted leakage. In particular, the system must ensure that the contents of any given ballot cannot be reconstructed, inferred, or correlated with a specific voter.

Auditability: It must be possible to audit the result of the election. In other words, one must be able to verify that the ballot tabulation happened correctly, and that the correct winner was chosen. Providing such auditability mechanisms is, however, outside the scope of this project.

Verifiability: A voter must be able to verify that their ballot has been counted, and that it has been counted correctly.

Ensuring individual or universal verifiability is likewise considered out of scope for this work.

5.3 Leakage Functions

In any voting system it is essential to clearly define what information or leakage is permitted to be revealed during or after tallying. The only strictly needed information is the output of the election, that is the winner. However, in many electoral contexts, leakage such as margins, turnout, or geographical breakdowns can also be desirable. Real world elections often publish such statistics [7]. Consequently, an election system should be accompanied by a well-specified leakage model that captures exactly what information is allowed to be learned. Below, we have listed some of the leakage functions that might be desirable in a real world system.

Minimal leakage/Winner only: This is the most restrictive leakage function, where only the winner of the election is revealed and nothing else.

Order of elimination: In a round based election it might be desirable to learn in which order the candidates are eliminated. While this information is implied if per-round vote totals are disclosed, it may be desirable to reveal only the elimination order and nothing else. This provides some transparency into the structure of the election without leaking full intermediate tallies.

Number of blank ballots: It may be desirable to learn the number of voters who submitted blank ballots.

Number of votes per candidate: In some cases, it can be desirable to know how many votes both the winner and every other candidate got. In plurality voting, the leakage would simply be the number of votes each candidate got. In RCV this would include leaking the number of votes each candidate got in each round. Another solution would be to leak the number of votes each candidate got for only the first x rounds.

Voter identities: It might be desirable to leak the identities of those who voted in the election. Note that learning who voted does not reveal who they voted for.

Voting statistics: In many elections, it is common to publish aggregated statistics on the distribution of votes. For instance, in the United States, results are frequently presented as a map distinguishing between states won by each major party. In Denmark, it is possible to view the distribution of votes at the level of individual polling stations[7]. This level of detail is feasible because

certain information becomes discernible once ballots are counted. Although this disclosure is a natural consequence of the counting process if counting happens at the polling stations, it is generally considered acceptable and often desirable in many electoral systems.

Depending on the context of a given election, any of the above leakage functions may be considered acceptable or even necessary. Different electoral systems place different emphasis on transparency, auditability, and privacy, and so the appropriate leakage profile must be chosen accordingly.

6 Solution Using an MPC Framework

To solve the problem of how to run a secure RCV election with an arbitrary leakage function, we settled on stating the problem in terms of addition and multiplication for integers and as OR and AND gates for boolean operations. It is known that any function represented in terms of addition and multiplication operations can be evaluated securely using MPC [8, 9].

Our design is described in the rest of this section.

6.1 Data Representation

6.1.1 The ballot matrix B

In our construction, a ballot is represented as a matrix of size $C \times C$, where C is the number of candidates. The entries in a ballot matrix B are defined as follows:

$$B[i, j] = \begin{cases} 1 & \text{if candidate } j \text{ is the } i\text{th priority} \\ 0 & \text{otherwise} \end{cases}$$

This means that there are up to C entries with a 1, and all other entries are 0. Additionally, no row or column cannot contain more than one 1. It is allowed to only rank a subset of the candidates.

Representing the ballots as matrices in this way allows us to represent most of the computation as simple arithmetic operations, which makes implementing the scheme within MPC both much easier, and much faster, as we can avoid all branching related to removing eliminated candidates and removing the non-highest priority candidates.

6.1.2 Active candidates vector A

A is a vector tracking the active candidates, i.e. those who are not yet eliminated:

$$A[i] = \begin{cases} 0 & \text{if candidate } i \text{ is eliminated} \\ 1 & \text{otherwise} \end{cases}$$

6.1.3 Round result vector V

V is computed as the sum of the votes in each round.

6.2 Ballot Input

In order to preserve the privacy of ballots, the client secret shares the ballot before submitting it to the servers. This ensures that no adversarial server can reconstruct the ballot, and thus learn the voter’s preference.

Any secret sharing scheme can be used, as long as the ballots can be reconstructed inside the MPC framework used by the servers. We have used additive secret sharing modulo 2 [10, page 11–14]

6.3 Computation

The computation of the election result consists of two major phases; ballot reconstruction and ballot counting.

6.3.1 Ballot reconstruction

Each server inputs its received ballot share into the MPC framework. The reconstruction computation is then carried out on these secret-shared values, so that the MPC protocol operates on a consistent representation of the original ballot without any server ever seeing the complete ballot or being able to reconstruct it on its own. In our implementation, we use additive secret sharing modulo 2, since all entries in the ballot matrices are bits.

Due to limitations in the MP-SPDZ client, each server currently secret shares its input share to the MPC as a workaround. The servers then perform a modular sum of these shares (mod 2), resulting in a secret-shared version of the original bit that can be used in subsequent MPC computations.

6.3.2 Ballot counting

Counting of the reconstructed ballots takes place using the circuit in figure 2. A round of counting takes as input all the ballots, as well as a vector denoting the candidates that are active at this time. Then the processes described below happen, after which one entry in the active candidates vector is changed, corresponding to the eliminated candidate being removed.

Remove Eliminated Candidates: For each entry in ballot B compute:

As integers:

$$B_{active}[i, j] = B[i, j] \cdot A[i]$$

As bits:

$$B_{active}[i, j] = B[i, j] \wedge A[i]$$

That is if candidate i is eliminated and this $A[i] = 0$ then the i th row is set to all zeroes. If not it remains the same.

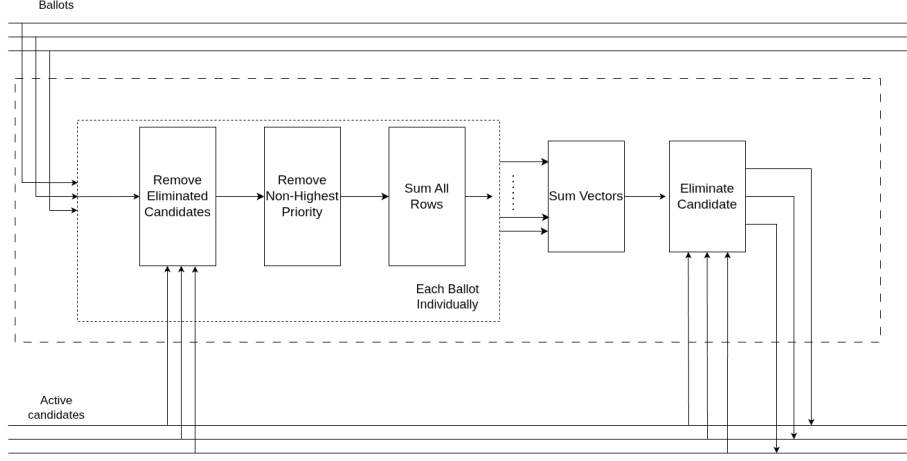


Figure 2: One round of ballot counting

Remove Non-Highest Priority: For each entry in ballot B_{active} compute:
As integers:

$$B_{highest}[i, j] = B_{active}[i, j] \cdot \left(\prod_k^{i-1} \left(1 - \sum_j B_{active}[k, j] \right) \right)$$

As bits:

$$B_{highest}[i, j] = B_{active}[i, j] \wedge \left(\bigwedge_{k \in [1, i-1]} \left(\neg \left(\bigvee_{j \in [1, C]} B_{active}[k, j] \right) \right) \right)$$

That is, for each column, set it to all 0 if and only if there exists a column to the left that contains a 1. After this process occurred, either the ballot is contains only 0s or there is exactly one 1 in the entire matrix, corresponding to the highest ranked candidate who has not yet been eliminated.

Sum All Rows: For a ballot $B_{highest}$ compute a vector v :

As integers:

$$v[j] = \sum_{i \in [1, C]} B_{highest}[i, j]$$

As bits:

$$v[j] = \bigvee_{i \in [1, C]} B_{highest}[i, j]$$

For a ballot $B_{highest}$ sum all the rows and output a vector of the sums. There is at most one 1 in this vector. If there is a 1 at position i then the ballot lists candidate i as its highest non eliminated candidate.

Sum Vectors: Given a list of vectors $v_1, \dots, v_n$ compute:

$$V[j] = \sum_{i=1}^C v_i[j]$$

Given a list of vectors sum them entrywise and output the vector V . $V[i]$ corresponds to how many votes candidate i got in this round.

Eliminate Candidate: To eliminate candidate j , update A as

$$A[j] = 0$$

For a candidate j update $A[j]$ to 0 indicating candidate j is no longer active.

6.4 Circuit Complexity

The complexity of the gates in the circuit will be expressed in terms of addition, multiplication and majority gates.

Remove Eliminated Candidates In order to remove any eliminated candidates from the ballot matrix each entry in the matrix is multiplied with $A[i]$ for the i 'th candidate. Thus making the complexity for a single ballot in a single round:

$C^2 = O(C^2)$ multiplication gates.

Remove Non-Highest Priority: In order to remove any candidate that is not the highest priority from a single column, we must compute the sum of all the columns to the left of this column. Since we must do this for each column, we get the complexities listed below for a single ballot in a single round:

$C^2 \cdot \frac{C(C-1)}{2} = O(C^4)$ multiplication gates.

$C \cdot \frac{C(C-1)}{2} = O(C^3)$ addition gates.

Sum All Rows: To sum all the rows in a matrix $C \cdot (C - 1)$ addition gates are needed, thus for a single ballot the complexity is:

$C \cdot (C - 1) = O(C^2)$ addition gates.

Sum Vectors: To sum n vectors, each of length C , we require:
 $C \cdot (n - 1)$ addition gates.

Eliminate Candidate: To eliminate a candidate the implemented code runs a for loop with an if statement and the complexity in terms of addition and multiplication will not be expressed, instead we will denote the complexity of finding the candidate to eliminate as $O(E)$. Thus to find and eliminate a candidate takes $O(E)$ and 1 multiplication gate to flip the active status.

6.5 Correctness of the RCV Tally

We argue that the tallying procedure outlined in Section 6.3.2 computes exactly the same outcome as the procedure defined in Section 3.2. The arguments proceed by showing that each round of the secure computation faithfully simulates one round of a classical RCV tally, and that the following set of invariants are preserved throughout the execution.

Ballot validity invariant: This is satisfied by assumption, recall our assumption with regards to a well formatted ballot, we assume that a ballot is a $C \times C$ matrix with at most one entry in each row and each column is equal to 1 and 0 for all other entries. At the beginning of each round we use the original ballot and not the one modified in the subprocess.

Active candidate invariant At the beginning of each round the vector A correctly encoded the set of candidates that has not yet been eliminated. That is

$$A[i] = \begin{cases} 0 & \text{if candidate } i \text{ is eliminated} \\ 1 & \text{if candidate } i \text{ is active} \end{cases}$$

In each round exactly one candidate, the one with the lowest vote count is eliminated by setting the corresponding entry in A to 0. No other operations are done for the vector A and the number of active candidates strictly decreases unless a winner is found.

Note that if two candidates are tied for lowest vote count use some tie breaking rule, for simplicity we assume the candidate with the lowest index is eliminated.

Correct vote distribution: For a ballot B to be counted correctly in a given round it must contribute exactly one vote to the highest-ranked candidate that has not been eliminated, and no vote to anyone if no ranked candidate is active. The tallying circuit ensures this by first removing all candidates that are no longer active for the ballot B . As described in section 6.3.2 for B_{active} where B_{active} is a copy of B where an entry $B_{active}[i, j]$ is set to 0 if either the entry itself is 0 or if candidate i is eliminated, i.e if candidate i was not the j th priority, or candidate i is eliminated the entry is set to 0 and otherwise 1. The ballot B_{active} is now identical to the original ballot except for row i if candidate i is eliminated.

Second, the circuit removes all but the highest-ranked active candidate. This is achieved by computing, for each entry $B_{highest}[i, j]$, the expression.

$$B_{highest}[i, j] = B_{active}[i, j] \cdot \left(\prod_k^{i-1} \left(1 - \sum_j B_{active}[k, j] \right) \right)$$

The product evaluates to 1 if and only if there exists no active candidate in any column $k > i$. Consequently, an entry $B_{highest}[i, j]$ can remain 1 only if

it corresponds to the highest ranked active candidate in the ballot. All lower-ranked entries are set to 0.

After these two steps, the resulting ballot matrix contains at most a single nonzero entry, corresponding exactly to the highest-ranked non-eliminated candidate, or consists entirely of zeroes if no ranked candidate remains active. Thus each ballot contributes at most one vote per round, and vote distribution matches the rules of ranked choice voting.

At last this matrix can be compressed to a vector of length C by summing the value of each row in the matrix $B_{highest}$. This results in a vector with at most one 1 at index i if candidate i was ranked highest and remains active, otherwise the vector is all zeroes.

Summing vectors: Given a set of vectors outputted by the voting distribution they are summed entry wise and are trivially correct.

Majority and termination conditions: Given the result of the summed vectors the protocol checks whether any active candidate has obtained a majority, or whether only a single candidate remains active. If either are true the protocol outputs the candidate as the winner. This is the termination condition for the classic RCV scheme. In the case where no information is leaked but the winner, this termination condition is not sufficient as one might be able to deduct some information based on the running time of the protocol or based on the data transmitted between servers. Thus, to ensure no extra information is leaked, the protocol runs all rounds. This does not change the result of the protocol, because any candidate who has a majority in round $\alpha - 1$ that candidate will also have a majority in round α .

6.6 Minimal leakage implementation

With our circuit in mind we decided to test our design by implementing it using MP-SPDZ . Our first of two implementations only reveals the winner. As previously mentioned we assumed that the ballots were preshared to the servers and thus our implementation reads each server’s input from its own file.

6.6.1 System security of minimal leakage

By employing secret shared ballots prior to submission to the servers, ballot secrecy is preserved, provided that fewer than the threshold number of servers are malicious. Under this assumption, no individual server, nor any coalition below the threshold, can reconstruct a voter’s ballot. When considering the minimal-leakage functionality, in which the only intended output is the identity of the winning candidate, additional care is required. In the current implementation, the servers necessarily learn the total number of ballots processed, and consequently the number of participating voters. As a result, the system leaks not only the winner, but also aggregate participation information. This additional leakage is not inherent to the functionality and can be mitigated.

Specifically, the number of ballots can be fixed in advance by padding the input with dummy ballots consisting entirely of zero entries, corresponding to blank votes that do not affect the election outcome. By processing a fixed number of ballots, the protocol ensures that no information beyond the identity of the winner is revealed.

Under this approach, the system achieves ballot secrecy and restricts information leakage to the minimal level required by the specified functionality, namely the election winner.

6.7 Round based leakage

The second of our implementations uses a less restrictive leakage function, namely leak the number of votes each candidate got in each round. Not only do we learn more about the distribution of votes but much of the computation can now be run outside MPC. That is, given how many votes each candidate got, we can easily compute who should be eliminated without needing MPC. As one might expect this provides a significant speedup when tallying the ballots, since some of the computations can take place in the clear, which is much faster than computing on secret shares. Besides the aforementioned benefits of this leakage profile, this implementation also provides information that is revealed in real world elections.

6.7.1 System security of more leakage

With respect to ballot secrecy, the same argument as previously discussed applies. The only additional information revealed in this setting is the aggregate number of votes received by each candidate in each round. This disclosure does not violate ballot secrecy, as the secrecy requirement concerns the inability to link an individual voter to a specific ballot choice. Given only the final tally, and assuming ballots remain secret, there is no mechanism to correlate any particular voter with a given vote. It is acknowledged that, in extreme cases, auxiliary information may enable limited inferences. For example, if a candidate receives exactly one vote and a voter knows that they cast that vote, the voter can deduce that no other voter selected that candidate. However, such inferences are inherent to any voting system that publishes election results and cannot be eliminated without suppressing outcome information entirely. Moreover, these edge cases have negligible practical impact in elections with a sufficiently large electorate. The publication of per-candidate vote counts and the total number of votes cast additionally reveals aggregate participation information, such as the number of blank ballots and overall voter turnout relative to the eligible electorate. In a real world election this information is commonly considered acceptable and is typically regarded as a feature rather than a vulnerability, as it contributes to transparency and public verifiability of the election process.

6.8 Why We Chose MP-SPDZ

We chose MP-SPDZ as the framework because it is a mature and well-established system that supports a wide range of MPC protocols with strong security guarantees. It provides built-in support for secret sharing, secure arithmetic and Boolean computation, and different adversarial thresholds, which makes it possible to implement complex functionalities without having to design or verify low-level cryptographic primitives.

A key reason for this choice is that MP-SPDZ supports the secure evaluation of arbitrary circuits, making it well suited for expressing the election functionality considered in this work. The framework handles details such as encryption, communication between parties, and consistency checks internally, which reduces the likelihood of implementation errors and allows the focus to remain on the protocol logic and security properties.

Finally, MP-SPDZ allows the security model to be selected as needed, for example whether an honest-majority or dishonest-majority setting is assumed, providing flexibility in both design and analysis [11, 12].

7 Experimental Results

After implementing the two versions of the RCV scheme, we ran some experiments in order to examine the impact of varying the number of voters, candidates and servers respectively. All experiments were run on the same machine:

- Model: Lenovo IdeaPad 5 Pro 16ACH6
- Processor: AMD Ryzen™ 5 5600H with Radeon™ Graphics $\times 12$
- OS: Ubuntu 24.04.3 LTS
- Memory: 16 GB

7.1 Varying number of voters

In figure 3, we have plotted the running times of a number of simulated elections, where the number of voters was varied across the simulations. The most immediately obvious result from the experiment is that tabulating the ballots is a lot faster if we allow the leakage of the vote ballot at the end of each round. Additionally, in the no leakage scenario, there is a very clear linear relationship between the number of voters, and the tabulation time of the ballots. This is exactly what we expect, as the amount of computation per ballot per round does not depend on the number of ballots cast. In the leakage scenario, the linear relationship is not quite as clear, and the variance of the running times at particular ballot numbers is a lot higher than in the no leaking scenario. The reason for this difference is that, in the leaking scenario, the ballot counting sometimes terminate before round $C - 1$, if a candidate has already reached a majority. Nevertheless, there is still very clearly a linear relationship between the number of ballots, and the tabulation time.

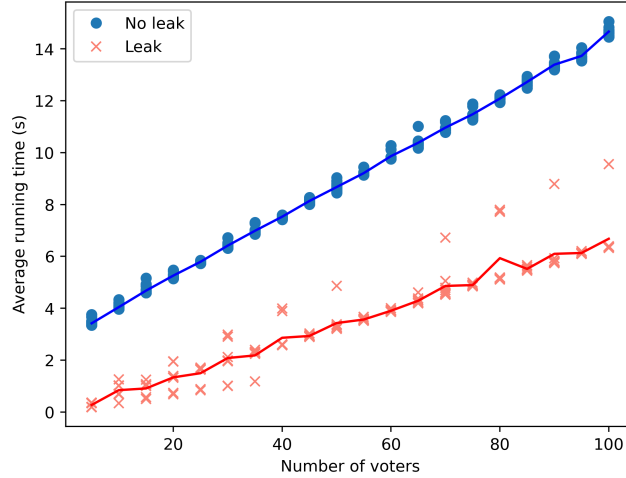


Figure 3: Running times of elections with varying numbers of voters. Each dot or cross represent a single election, and the lines represent the average running time of the elections of the given size.

7.2 Varying number of candidates

In figure 4 we show the results of simulating a number of elections with a varying number of candidates. We can see that, unlike before, there does not seem to be a linear relationship between the number of candidates, and the running time. In fact, there seems to be a quadratic relationship, most clearly in the no leakage scenario. In figure 5, we have plotted the same data as in figure 4, but have scaled the y-axis with C^{-2} . As this new plot shows a linear correlation, the real correlation between the number of candidates and the running time of the election is a quadratic.

This relationship is not what we expected. As described previously, the amount of operations required per ballot per round is $O(C^4)$, which would mean that the total running time of the entire protocol is $O(C^5)$, since there are $O(C)$ rounds. However, that theoretic running time does not translate to the concrete implementation. We believe that the reason for this is that we use MP-SPDZ, which employs parallelization, such that the time taken to complete the computation is faster than the theoretical design would suggest.

7.3 Varying number of servers

In figure 6, we show the running times of elections distributed across a varying number of servers, but with a fixed number of both candidates and voters. As in the other experiments, the leakage version consistently outperforms the

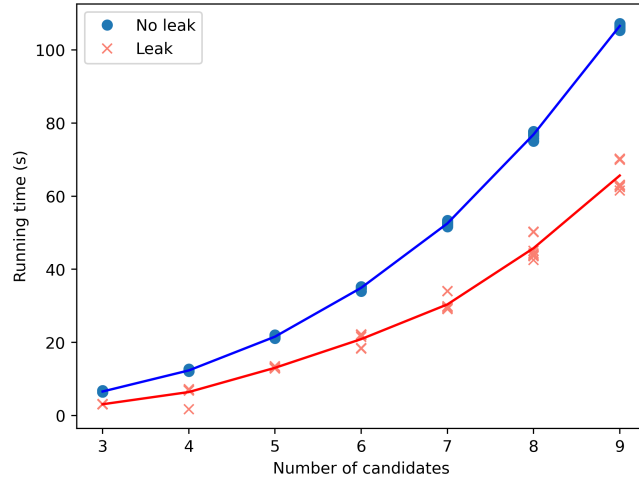


Figure 4: Running times of elections with varying numbers of candidates. Each dot or cross represent a single election, and the lines represent the average running time of the elections of the given size

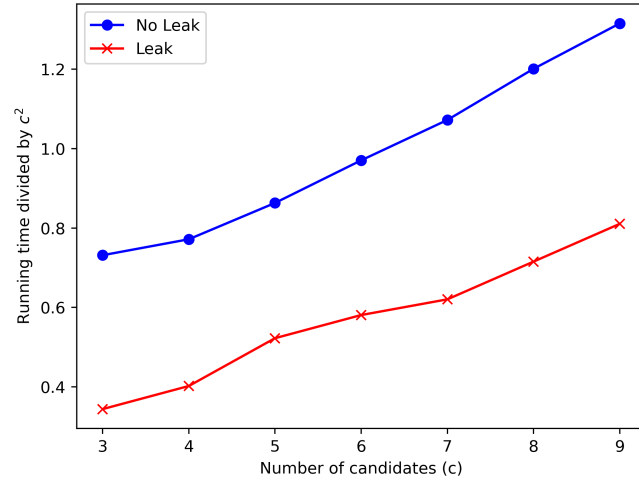


Figure 5: Same data as figure 4, but the y-axis is scaled by c^{-2}

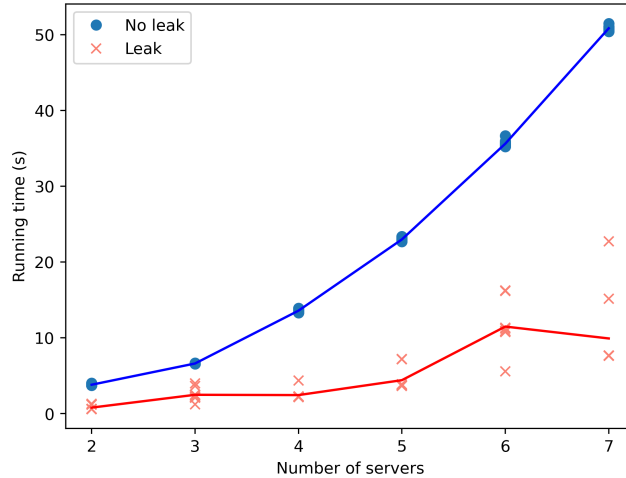


Figure 6: Running time of an election with 3 candidates, 32 voters, and with a varied number of servers. Each dot or cross is an individual election simulation, and the lines are the average running times.

non-leakage version.

In this experiment, we do not see similar shapes for the two versions, unlike in the other experiments. Specifically, while the non-leakage version seems to be some form of quadratic relationship, the leakage version is much flatter, especially when looking at the average running times.

The likely reason for the not-quite-linear relationship between the number of servers and the running time is due to the overhead required for maintaining the connections to more servers, as well as generally increased complexity of running MPC with more parties.

8 Limitations and Future work

While the proposed system demonstrates that RCV can be tallied securely using MPC, several limitations do remain. These limitations are largely the result of deliberate design choices made to keep the system tractable within the scope of this project.

Honest Clients and Ballot Formation: At this moment the system assumes that voters and the client interface are honest and respect the ballot formation required by the system. In a real world setting it would be naive to assume that all clients are always honest; a malicious client might either submit malformed ballots (with e.g. duplicate rankings), or might just drop all com-

munications. Addressing this would require additional mechanisms for ballot validation. One solution is for the client to prove to the servers that the ballot is well formed using a zero-knowledge proof. This is outside the scope of this project, but should be considered.

End-to-end Verifiability: Although the system guarantees correctness and ballot privacy under the defined threat model, it does not provide end-to-end verifiability. In particular, voters cannot independently verify that their ballot was included in the tally and counted correctly. Incorporating verifiable MPC or cryptographic audit mechanisms would improve transparency.

Ballot Representation: With the current ballot representation, the protocol runs in time polynomial in the number of candidates and linear in the number of voters, with a high-degree dependence on the number of candidates. While this is less than optimal, it is not a big problem, since the number of candidates tends to be limited in real-world elections. Future work could include figuring out a more space-efficient ballot replication.

Tie Breaking: In any voting system, in the unlikely case that two candidates have an equal number of votes, a tie breaking rule is required. While the current rule of using the candidate with the lowest index is by no means fair, this project's scope does not include making a fair tie breaking rule. It should most definitely be considered for a real election; such rules should be carefully specified by election authorities.

Fault Tolerance: The system assumes reliable communication over secure channels and that all tabulation servers remain online for the duration of the protocol. Issues related to server failures, network disruptions, or denial-of-service attacks are not addressed. Supporting fault tolerance and availability would require additional protocol mechanisms and is left for future work. For example, in a recent electronic election reported by IACR, one of the trustees lost their part of a threshold decryption key, making it impossible to recover the election outcome [13].

Despite these limitations, the system fulfills its intended goal of demonstrating that true ranked choice voting, including elimination and vote redistribution, can be implemented securely and efficiently using multiparty computation under well-defined assumptions.

References

- [1] John Bartholdi and James Orlin. Single transferable vote resists strategic voting. *Social Choice and Welfare*, 8:341–354, 01 1991. doi: 10.1007/BF00183045. URL <https://doi.org/10.1007/BF00183045>.
- [2] Carina I. Hausladen, Regula Hänggli Fricker, Dirk Helbing, Renato Kunz, Junling Wang, and Evangelos Pournaras. How voting rules impact legitimacy. *Humanities and Social Sciences Communications*, 11(1):689, May 2024. ISSN 2662-9992. doi: 10.1057/s41599-024-03056-8. URL <https://doi.org/10.1057/s41599-024-03056-8>.
- [3] Xuechao Yang, Xun Yi, Surya Nepal, Andrei Kelarev, and Han Fengling. A secure verifiable ranked choice online voting system based on homomorphic encryption. *IEEE Access*, PP:1–1, 03 2018. doi: 10.1109/ACCESS.2018.2817518. URL <https://doi.org/10.1109/ACCESS.2018.2817518>.
- [4] Xuechao Yang, Xun Yi, and Andrei Kelarev. Secure ranked choice online voting system via intel sgx and blockchain. In *2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 139–146, 2021. doi: 10.1109/TrustCom53373.2021.00036. URL <https://doi.org/10.1109/TrustCom53373.2021.00036>.
- [5] J. J. Bartholdi, C. A. Tovey, and M. A. Trick. The computational difficulty of manipulating an election. *Social Choice and Welfare*, 6(3):227–241, Jul 1989. ISSN 1432-217X. doi: 10.1007/BF00295861. URL <https://doi.org/10.1007/BF00295861>.
- [6] United Nations. International covenant on civil and political rights. <https://www.ohchr.org/en/instruments-mechanisms/instruments/international-covenant-civil-and-political-rights>, 1966. Adopted 16 December 1966, entered into force 23 March 1976, Article 25.
- [7] Danmarks Statistik. National elections in denmark, 2025. URL <https://www.dst.dk/en/Statistik/dokumentation/documentationofstatistics/national-elections-in-denmark>.
- [8] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing*, STOC ’88, page 1–10, New York, NY, USA, 1988. Association for Computing Machinery. ISBN 0897912640. doi: 10.1145/62212.62213. URL <https://doi.org/10.1145/62212.62213>.
- [9] O. Goldreich, S. Micali, and A. Wigderson. How to play any mental game. In *Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing*, STOC ’87, page 218–229, New York, NY, USA,

1987. Association for Computing Machinery. ISBN 0897912217. doi: 10.1145/28395.28420. URL <https://doi.org/10.1145/28395.28420>.
- [10] Ronald Cramer, {Ivan Bjerre} Damgård, and {Jesper Buus} Nielsen. *Secure Multiparty Computation and Secret Sharing*. Cambridge University Press, 2015. ISBN 9781107043053.
 - [11] Marcel Keller. MP-SPDZ: A versatile framework for multi-party computation. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020. doi: 10.1145/3372297.3417872. URL <https://doi.org/10.1145/3372297.3417872>.
 - [12] Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. Multi-party computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, pages 643–662, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg. ISBN 978-3-642-32009-5.
 - [13] IACR News. Trustee loses key in electronic election, making outcome irrecoverable, 2025. URL <https://www.iacr.org/news/item/27138>. Accessed: 2025-12-18.

A Implementation code

Our code for the implemented schemes is available here:

🔗 <https://github.com/MikkelKatholm/Rank-Voting-Project>